

14) Testing Quality and CICD

Last updated by | Mennell, Jacob | 19 May 2026 at 15:42 BST

Testing, Quality, And CI/CD

Purpose

This page describes how the solution validates code quality, runs tests, and packages deliverables through CI/CD processes.

Scope

The build pipeline treats testing and analysis as first-class concerns. Validation is split across backend services, shared libraries, frontend build steps, and static quality analysis.

Test Organization

The repository includes test suites for:

- API service
- main worker
- ingestion worker
- aiutil shared retrieval library

This reflects the actual service and library boundaries rather than treating the backend as one monolithic test target.

Coverage Strategy

The build pipeline runs each major Python test slice with its own coverage output. Coverage is then published and also supplied to SonarCloud.

This keeps visibility on shared-library quality, especially for retrieval behavior, rather than allowing only service-level tests to dominate the signal.

Build Pipeline Stages

The pipeline performs the following major steps:

1. prepare runtime prerequisites such as Python and Node.js
2. install backend dependencies in editable mode across services and libraries
3. run unit tests and collect coverage for API, main worker, aiutil, and ingestion worker
4. publish test results and coverage reports
5. install frontend dependencies and build the frontend
6. run SonarCloud analysis
7. build Python packages for the backend modules

Together, these steps produce a combined quality signal covering correctness, coverage, and code quality analysis.

Quality Gates And Signals

The pipeline emits several useful quality signals:

- per-slice unit test results
- per-slice coverage reports
- frontend build success
- SonarCloud analysis and publication

The presence of dedicated aiutil coverage and tests is particularly important because retrieval behavior is a central dependency for the AI workflows described elsewhere in the wiki.

CI/CD Flow Diagram



flowchart TD

```

Commit[Code Change] --> Install[Install Dependencies]
Install --> ApiTests[API Tests]
Install --> MainTests[Main Worker Tests]
Install --> AiutilTests[aiutil Tests]
Install --> IngestTests[Ingestion Worker Tests]
ApiTests --> Coverage[Publish Coverage and Test Results]
MainTests --> Coverage
AiutilTests --> Coverage
IngestTests --> Coverage
Install --> FrontendBuild[Frontend Build]
Coverage --> Sonar[SonarCloud Analysis]
FrontendBuild --> Sonar
Sonar --> Package[Build Python Packages]
  
```

Design Decisions

- Shared libraries are tested explicitly rather than only indirectly through services.
- Coverage is collected per slice, which makes gaps easier to reason about.
- Frontend build validation is included in the same pipeline as backend validation.
- Static analysis is integrated into the build rather than treated as an optional afterthought.

Operational Notes

- A passing build means more than just successful packaging; it also implies a baseline level of test and analysis validation.
- Pipeline output can help localize regressions to a service or shared library boundary.
- The current pipeline is a strong source of externally shareable evidence that engineering quality is managed in a structured way.